

Programmazione Concorrente e Distribuita

Ingegneria e Scienze Informatiche - UNIBO

a.a 2025/2026

Prof. Alessandro Ricci

[module 1.1]

CONCURRENT PROGRAMMING INTRODUCTION

SUMMARY

- Concurrent programming
 - motivations: HW evolution
 - basic jargon
 - processes interaction, cooperation, competition,
 - mutual exclusion, synchronisation
 - problems: deadlocks, starvation, livelocks
- Concurrent languages, mechanisms, abstractions
 - overview

CONCURRENCY AND CONCURRENT SYSTEMS

- Concurrency as a main concept of many domains and systems
 - operating systems, multi-threaded and multi-process programs, distributed systems, control systems, *real-time* systems,...
- General definitions
 - “*In computer science, concurrency is a property of systems in which several computational processes are executing **at the same time**, and potentially **interacting** with each other.*” [ROS-97]
 - “*Concurrency is concerned with the fundamental aspects of systems of multiple, **simultaneously active** computing agents, that **interact** with one another*” [CLE-96]
- Common aspects
 - systems with multiple activities or **processes** whose execution **overlaps in time**
 - activities can have some kind of *dependencies*, therefore can **interact**

CONCURRENT PROGRAMMING

- **Concurrent programming**
 - building programs in which multiple computational activities overlap in time and typically interact in some way
- **Concurrent program**
 - finite set of *sequential* programs that can be executed in parallel, i.e. overlapped in time
 - a sequential program specifies sequential execution of a list of statements
 - the execution of a sequential program is called **process**
 - a concurrent program specifies two or more sequential programs that may be executed concurrently as *parallel processes*
 - the execution of a concurrent program is called *concurrent computation or elaboration*

TERMINOLOGY

- **Concurrent** programming
 - building programs in which multiple computational activities *overlap* in time and typically interact in some way
 - without necessarily running on separate physical processors
 - logical/abstract/programming level
 - focus on program organization
- **Parallel** programming
 - the execution of programs overlaps in time by running on *separate physical processors*
 - physical level
 - focus on performance
- **Distributed** programming
 - when processors are distributed over a network
 - no shared memory

CONCURRENT VS. PARALLEL

- Rob Pyke's talk about "Go Concurrency Patterns" [*]
 - world as set of interacting agents
 - this is not captured by sequential programming paradigms
 - concurrency as the composition of independently executing computations.
 - it's a way of *structuring software*, to write clean code that interacts well with the real world.
 - it's a model for software construction
 - easy to understand, use, reason about
 - *It is not parallelism. it enables parallelism*
 - parallelism => algorithmic view, performance

(*) <https://www.youtube.com/watch?v=f6kdp27TYZs>

CONCURRENCY “PARADIGMS”

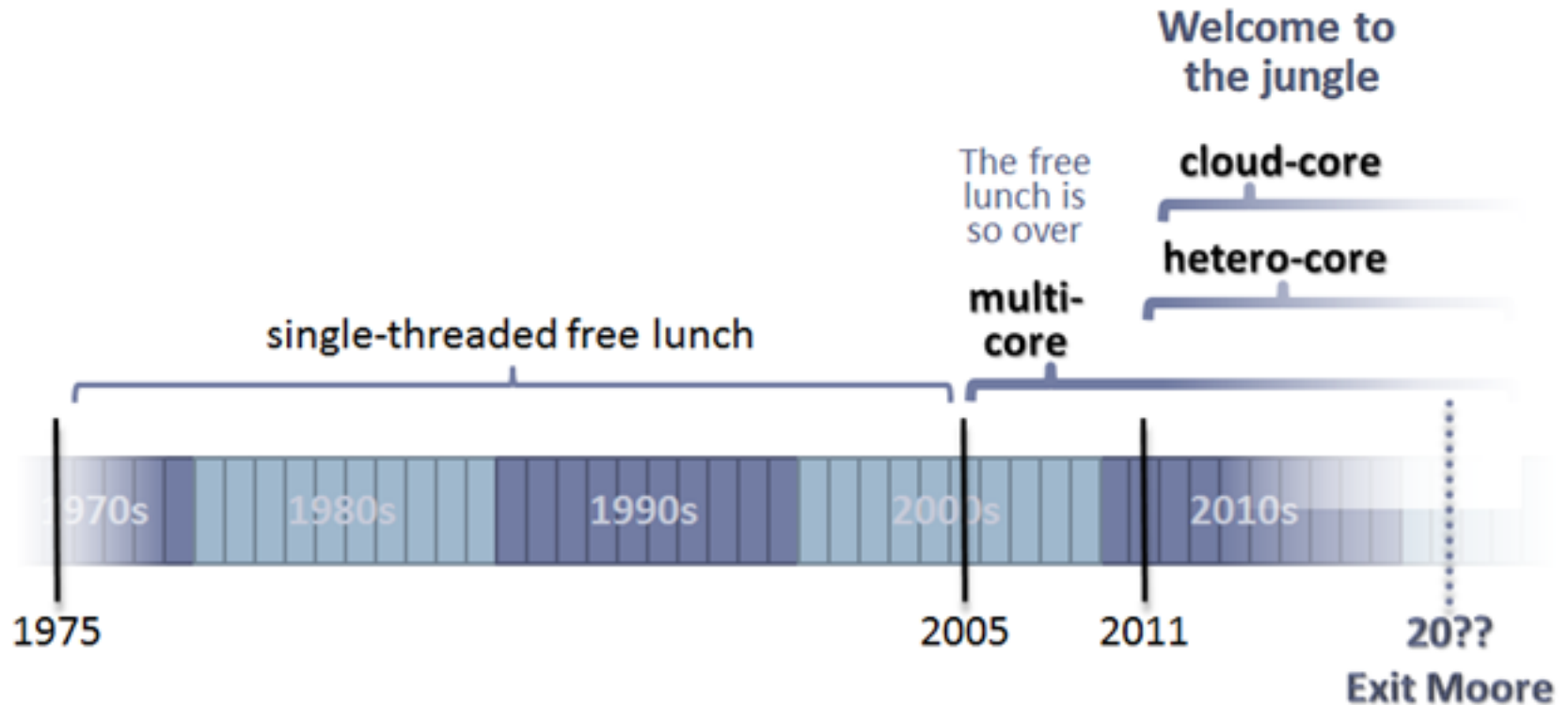
- **Multi-threaded programming**
 - shared state
 - synchronisation mechanisms
 - semaphores, monitors
- **Message-based programming**
 - no shared state
 - interaction by means of message exchange
 - *asynchronous* vs. *synchronous*
- **Event-driven programming**
 - the flow of the program is determined by events
 - user actions (mouse clicks, key presses), sensors, messages from other threads/process/apps

CONCURRENCY “PARADIGMS”

- **Asynchronous programming**
 - designing programs featuring asynchronous actions and requests
 - never blocking dogma
 - future mechanisms, callbacks
- **Reactive programming**
 - the flow of the program is designed around data flows and the propagation of change

PARALLEL COMPUTERS: “THE HARDWARE (CORE) JUNGLE”

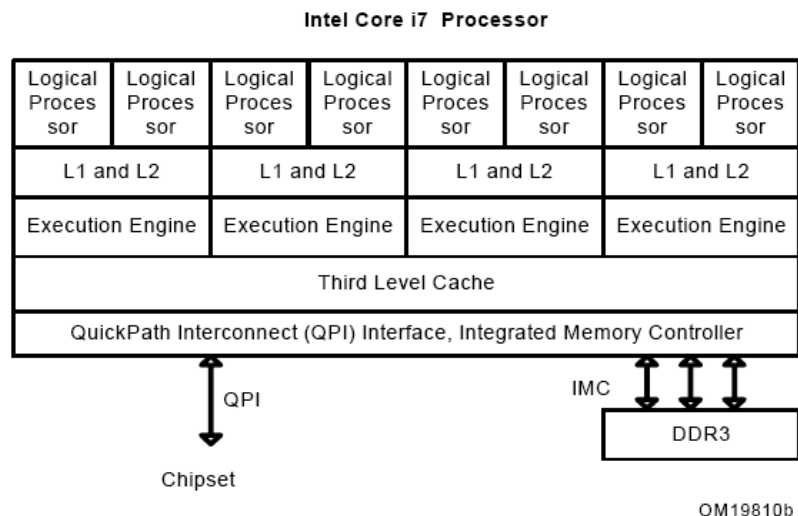
- *“The Free Lunch is Over. Now Welcome to the Hardware Jungle”*
(Herber Sutter, [SUT-12])



MULTI-CORE ARCHITECTURES

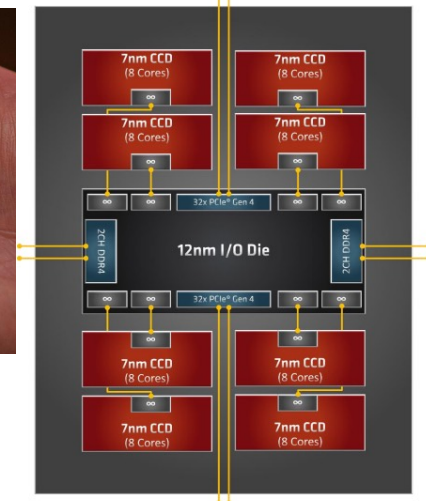
- **Chip multiprocessors - Multicore**
 - multiple cores on a single chip
 - sharing RAM, possibly sharing cache levels
 - examples: Intel Core i7, AMD Rizen Threadripper 3990X

Intel Core i7 family, ~2009



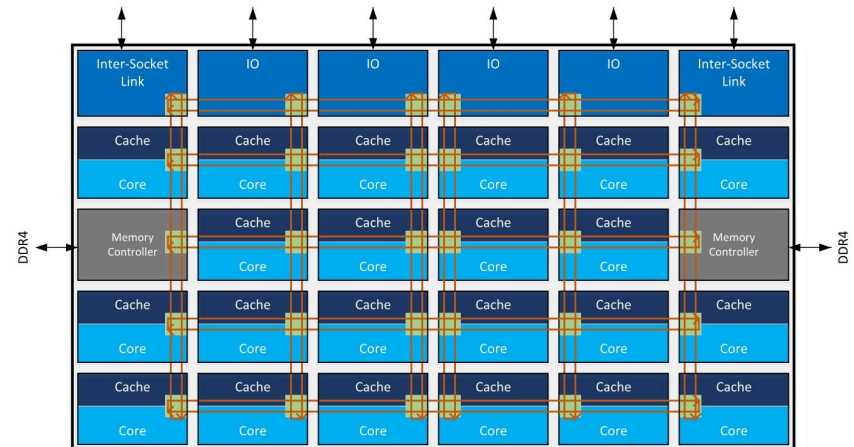
AMD Ryzen™ Threadripper™ 3990X Processor (2020)

- 64-bit **64-core** high-performance x86 desktop microprocessor
- Base frequency of 2.9 GHz with a TDP of 280 W and a boost of up to 4.3 GHz.
- Supports up to 512 GiB of quad-channel DDR4-3200 memory.



MULTI-CORE HYBRID ARCHITECTURES

- Recent architecture are *hybrid*, combining CPU cores of varying sizes
 - Performance-cores (P-cores)**
 - traditional design, featuring high clock speeds
 - Efficient-cores (E-cores)**
 - slower but physically smaller, consume much less power
- Example:
 - Intel Core i9 - 14th Gen Intel® Core™ desktop processors - 24 cores (up to 32 threads)
 - 8 P-cores
 - highest-performing CPU core, 5.8 GHz
 - 16 E-cores
 - for background tasks



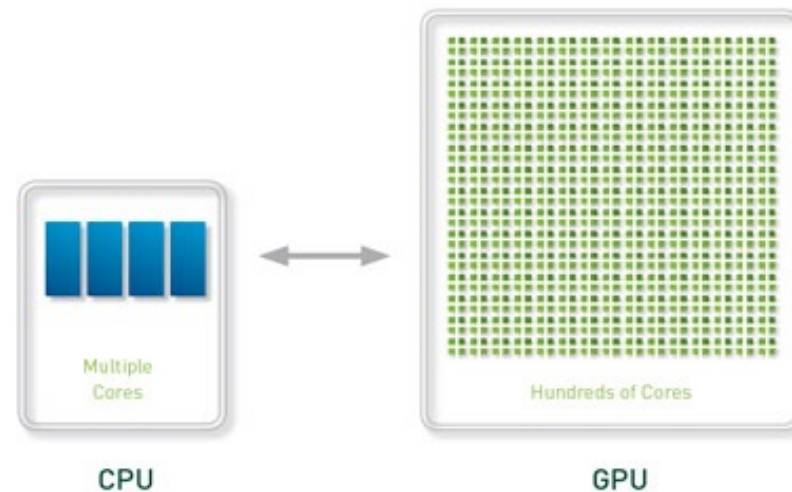
MULTI-CORE HYBRID ARCHITECTURES

- Another example: **AMD Zen Core Architecture - last gen: Zen 5**
 - Ryzen (consumer) and EPYC (server) series
 - scalability and high multi-core performance
- Key Aspects of Modern AMD Architecture: Chiplet Design
 - instead of one large silicon die, AMD uses "chiplets" (Core Complex Dies or CCDs) connected by a high-speed fabric, allowing for higher core counts
 - up to 16 cores/32 threads on desktop, 128+ on server
- In a die we can have either P-Cores or E-Cores
 - “Zen 5” — core is optimized for high performance
 - “Zen 5c” — core optimized for density and efficiency

PARALLEL COMPUTERS: HETEROGENEOUS CORES & MANY-CORE

- **Heterogeneous Chips Designs**

- augmenting a standard processor with one or more specialised compute engines, called *attached processors*
 - examples: Graphical Processing Units (GPU), **GPGPU** (General-Purpose Computation on Graphics Hardware), Field-Programmable Gate Array (FPGA), Cell processors, **CUDA** architecture

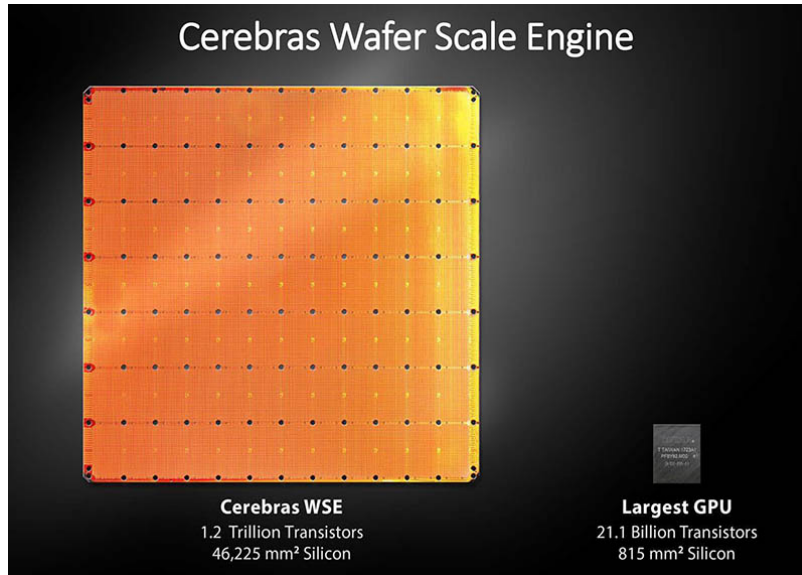


PARALLEL COMPUTERS: HETEROGENEOUS CORES & MANY-CORE

- Example: **Apple Silicon M5**
 - **SoC** - including CPU, GPU, NPU (neural processing unit) used for machine learning), ISP (coprocessor for elaborating images from photo sensor), Mx (coprocessor for elaborating data from integrated sensors), SEP (data protection)
 - Up to 10 CPU cores
 - 3 or 4 P-cores and 4 or 6 E-cores
 - Up to 10 GPU cores

PARALLEL COMPUTERS: DESIGNED FOR AI

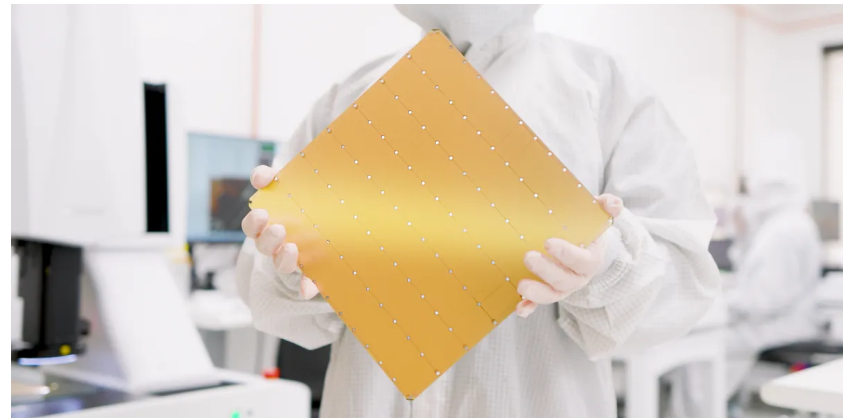
- E.g. Cerebras — latest CS-3 system (2024)



Wafer Scale Engine-3 vs. H100			
	Cerebras WSE-3	Nvidia H100	Cerebras Advantage
Chip size	46,225 mm ²	814 mm ²	57x
Cores	900,000	16,896 FP32 + 528 Tensor	52x
On-chip memory	44 Gigabytes	0.05 Gigabytes	880x
Memory bandwidth	21 Petabytes/sec	0.003 Petabytes/sec	7,000X
Fabric bandwidth	214 Petabits/sec	0.0576 Petabits/sec	3,715X



PCD LM - ISI - Cesena - UNIBO



Concurrent Programming Overview

PARALLEL COMPUTERS: SUPER-COMPUTERS

- Traditionally used by national labs and large companies
- Different kind of architectures, including clusters
- Typically large number of processors (multi-core) connected by ad-hoc networks
 - example: Fugaku Super computer

RIKEN Center for Computational Science

Manufacturer: Fujitsu

Cores: 7,630,848

Memory: 5,087,232 GB

Processor: A64FX 48C 2.2GHz

Interconnect: Tofu interconnect D



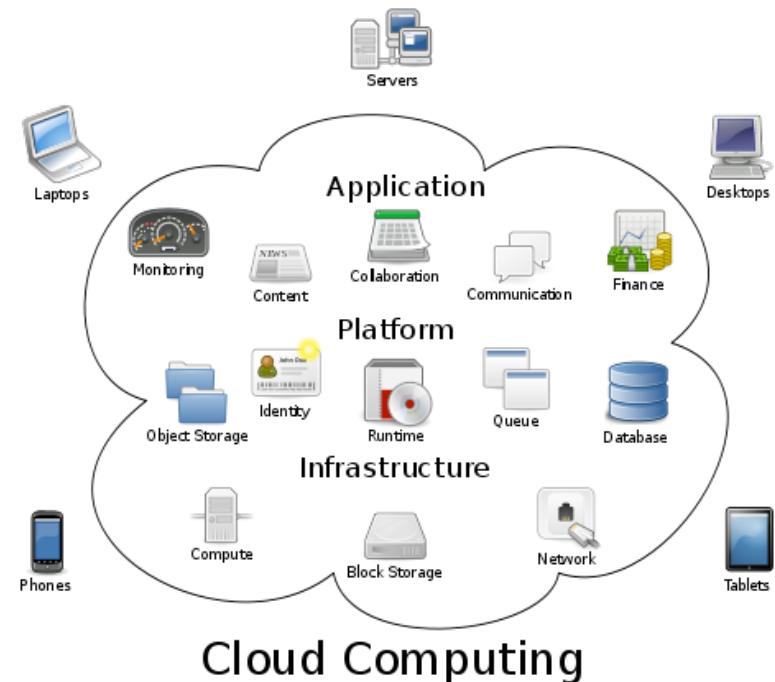
PARALLEL COMPUTERS: CLUSTERS / GRID

- Made from commodity parts
 - nodes are boards containing one or few processors, RAM and sometimes a disk storage
 - nodes connected by commodity interconnect
 - e.g. Gigabit Ethernet, Myrinet, InfiniBand, Fiber Channel
- Memory not shared among the machines
 - processors communicate by message passing
- Examples
 - System X supercomputer at Virginia Tech, a 12.25 TFlops computer cluster of 1100 Apple XServe G5 2.3 GHz dual-processor machines (4 GB RAM, 80 GB SATA HD) running Mac OS X and using InfiniBand interconnect
- Grid computing



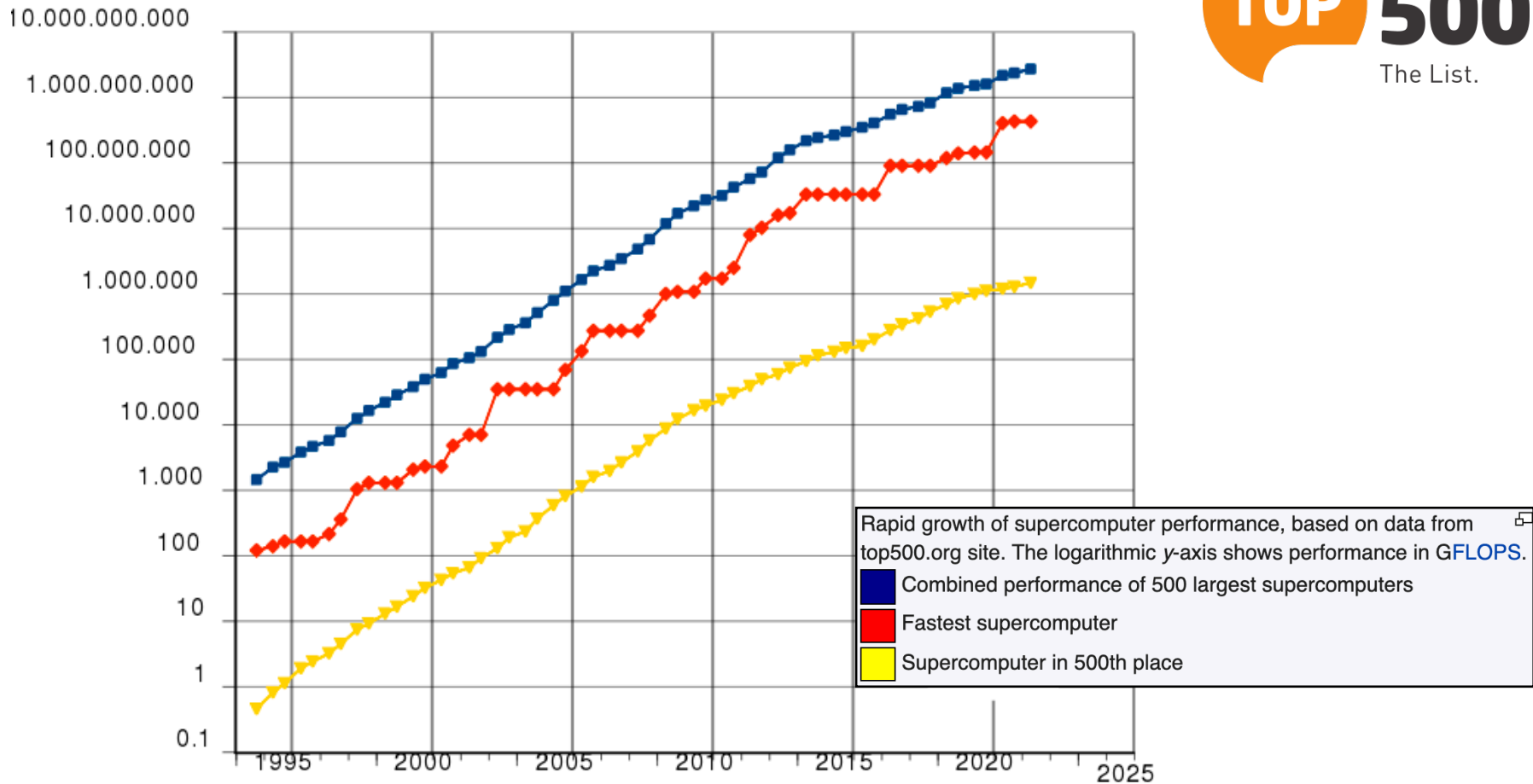
PARALLEL COMPUTERS: CLOUD COMPUTING

- Delivering computing **as a service** through the network
 - shared resources, software, and information are provided to computers and other devices as a metered service over a network (typically the Internet)
- X as a Service
 - Software as a Service (SAAS)
 - Platform as a Service (PAAS)
 - Infrastructure as a Service (IAAS)
- Public clouds, private clouds
- Examples
 - Amazon EC2 (Elastic Computing Cloud)
 - Microsoft Azure, Google App Engine



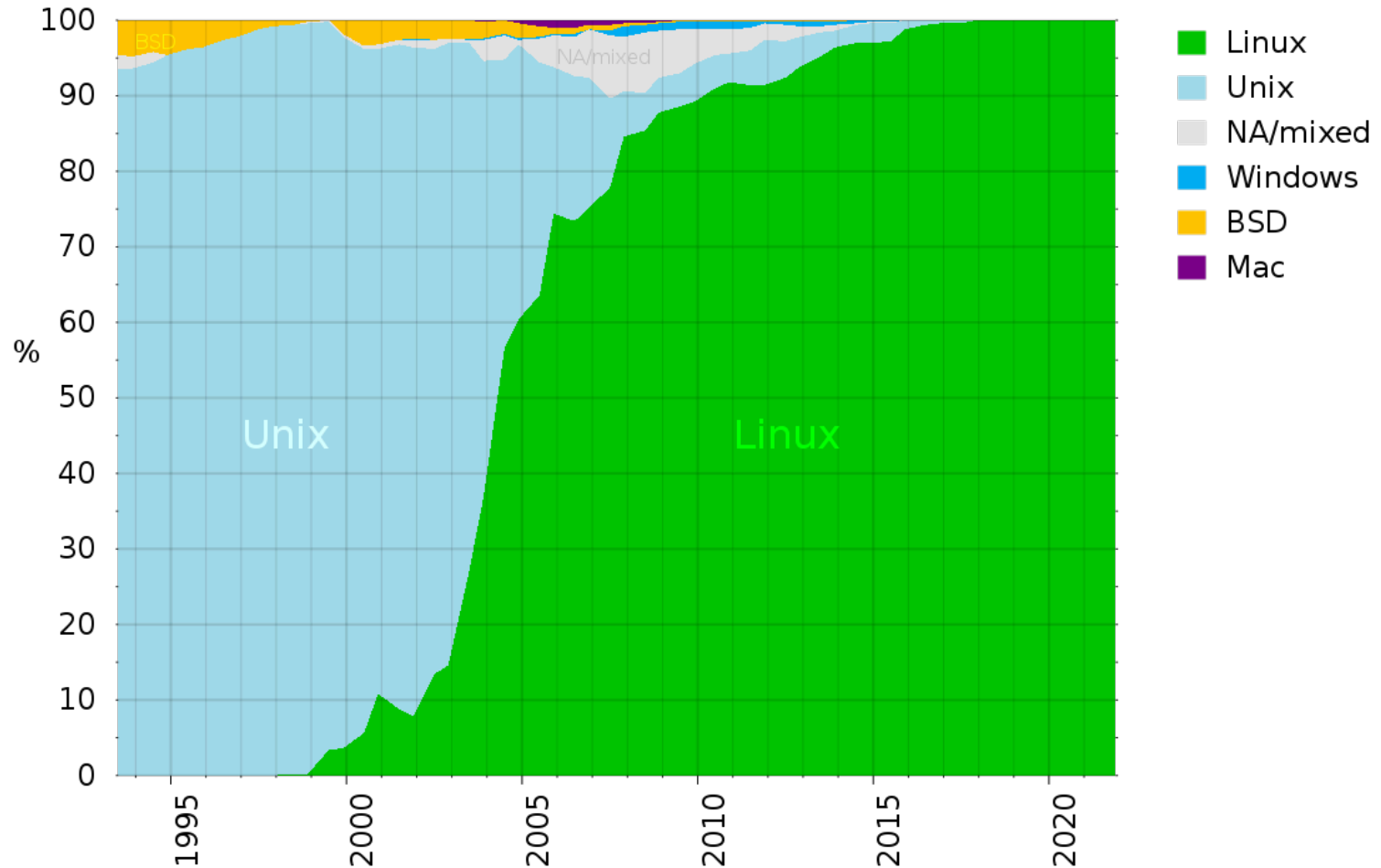
TOP 500 - FASTEST SUPERCOMPUTERS

- <https://en.wikipedia.org/wiki/TOP500>



TOP 500 - FASTEST SUPERCOMPUTERS

- OS kernels used in supercomputers



<https://en.wikipedia.org/wiki/TOP500>

THE FASTEST (Nov. 2025)

<https://en.wikipedia.org/wiki/TOP500>

Top 10 positions of the total 10,000 in November 2025

Rank (previous)	Rmax Rpeak (PetaFLOPS)	Name	Model	CPU cores	Accelerator (e.g. GPU) cores	Total cores (CPUs + accelerators)	Interconnect	Manufacturer	Site country	Year	Operating system
1 ▲	1,809.00 2,821.10	El Capitan	HPE Cray EX255a	1,080,000 (45,000 × 24-core Optimized 4th Generation EPYC 24C @1.8 GHz)	10,260,000 (45,000 × 228 AMD Instinct MI300A)	11,340,000	Slingshot-11	HPE	Lawrence Livermore National Laboratory 🇺🇸 United States	2024	Linux (TOSS)
2	1,353.00 2,055.72	Frontier	HPE Cray EX235a	614,656 (9,604 × 64-core Optimized 3rd Generation EPYC 64C @2.0 GHz)	8,451,520 (38,416 × 220 AMD Instinct MI250X)	9,066,176	Slingshot-11	HPE	Oak Ridge National Laboratory 🇺🇸 United States	2022	Linux (HPE Cray OS)
3	1,012.00 1,980.01	Aurora	HPE Cray EX	1,104,896 (21,248 × 52-core Intel Xeon Max 9470 @2.4 GHz)	8,159,232 (63,744 × 128 Intel Max 1550)	9,264,128	Slingshot-11	HPE	Argonne National Laboratory 🇺🇸 United States	2023	Linux (SUSE Linux Enterprise Server 15 SP4)
4 ▲	1,000.00 1,226.28	JUPITER	BullSequana XH3000	1,694,592 (23,536 × 72-Arm Neoverse V2 cores Nvidia Grace @3 GHz)	3,106,752 (23,536 × 132 Nvidia Hopper H100)	4,801,344	Quad-rail NVIDIA NDR200 Infiniband	Atos	EuroHPC JU 🇪🇺 European Union, Jülich, 🇩🇪 Germany	2025	Linux (RHEL)
5	561.20 846.84	Eagle	Microsoft NDv5	172,800 (3,600 × 48-core Intel Xeon Platinum 8480C @2.0 GHz)	1,900,800 (14,400 × 132 Nvidia Hopper H100)	2,073,600	NVIDIA Infiniband NDR	Microsoft	Microsoft 🇺🇸 United States	2023	Linux (Ubuntu 22.04 LTS)

THE FASTEST (Nov. 2025)

<https://en.wikipedia.org/wiki/TOP500>

6	477.90 606.97	HPC6	HPE Cray EX235a	213,120 (3,330 × 64-core Optimized 3rd Generation EPYC 64C @2.0 GHz)	2,930,400 (13,320 × 220 AMD Instinct MI250X)	3,143,520	Slingshot-11	HPE	Eni S.p.A European Union, Ferrara Erbognone, Italy	2024	Linux (RHEL 8.9)
7	442.01 537.21	Fugaku	Supercomputer Fugaku	7,630,848 (158,976 × 48-core Fujitsu A64FX @2.2 GHz)	-	7,630,848	Tofu interconnect D	Fujitsu	Riken Center for Computational Science Japan	2020	Linux (RHEL)
8	434.90 574.84	Alps	HPE Cray EX254n	748,800 (10,400 × 72-Arm Neoverse V2 cores Nvidia Grace @3.1 GHz)	1,372,800 (10,400 × 132 Nvidia Hopper H100)	2,121,600	Slingshot-11	HPE	CSCS Swiss National Supercomputing Centre Switzerland	2024	Linux (HPE Cray OS)
9	379.70 531.51	LUMI	HPE Cray EX235a	186,624 (2,916 × 64-core Optimized 3rd Generation EPYC 64C @2.0 GHz)	2,566,080 (11,664 × 220 AMD Instinct MI250X)	2,752,704	Slingshot-11	HPE	EuroHPC JU European Union, Kajaani, Finland	2022	Linux (HPE Cray OS)
10	241.20 306.31	Leonardo	BullSequana XH2000	110,592 (3,456 × 32-core Xeon Platinum 8358 @2.6 GHz)	1,714,176 (15,872 × 108 Nvidia Ampere A100)	1,824,768	Quad-rail NVIDIA HDR100 Infiniband	Atos	EuroHPC JU European Union, Bologna, Italy	2023	Linux (RHEL 8) ^[31]

FLYNN'S TAXONOMY

- Categorization of all computing systems according to the *number of instruction stream and data stream*
 - stream as a sequence of instruction or data on which a computer operate
- Four possibilities
 - **Single Instruction, Single Data (SISD)**
 - Von-Neumann model, single processor computers
 - **Single Instruction, Multiple Data (SIMD)**
 - single instruction stream concurrently broadcasted to multiple processors, each with its own data stream
 - fine grained parallelism, vector processors
 - **Multiple Instruction, Single Data (MISD)**
 - no well known systems fit this
 - **Multiple Instruction, Multiple Data (MIMD)**
 - each processor has its own stream of instructions operating on its own data

MIMD MODELS

- MIMD category can be then decomposed according to memory organization
 - **shared memory**
 - all processes (processors) share a single address space and communicate each other by writing and reading shared variables
 - **distributed memory**
 - each process (processor) has its own address space and communicate with other process by *message passing* (sending and receiving messages)

MIMD FURTHER CLASSIFICATIONS

- Two further classes for shared-memory computers
 - **SMP (Symmetric Multi-processing Architecture)**
 - all processors share a connection to a common memory and access all location memories at equal speed
 - **NUMA (Non-uniform Memory Access)**
 - the memory is shared, by some blocks of memory may be physically more closely associated with some processors than others

MIMD FURTHER CLASSIFICATIONS

- Two further classes for distributed-memory computers
 - **MPP (Massively Parallel Processors)**
 - processors and the network infrastructure are tightly coupled and specialized for a parallel computer
 - extremely scalable, thousands of processors in a single system
 - for High-Performance Computing (HPC) applications
 - **Clusters**
 - distributed-memory systems composed of off-the-shelf computers connected by an off-the-shelf network
 - e.g. Beowulf clusters (= clusters on Linux)
 - **Grid**
 - systems that use distributed, heterogeneous resources connected by LAN and/or by WAN, without a common point of administration

WHY CONCURRENT PROGRAMMING: PERFORMANCE

- **Performance improvement**
 - increased application **throughput**
 - by exploiting parallel hardware
 - increased application **responsiveness**
 - by optimizing the interplay among CPU and I/O activities
- Quantitative measurement for performance: **speedup**

$$S = \frac{T_1}{T_N}$$

N is the number of processors

T_1 is the execution time of the sequential algorithm

T_N is the execution time of the parallel algorithm with N processors

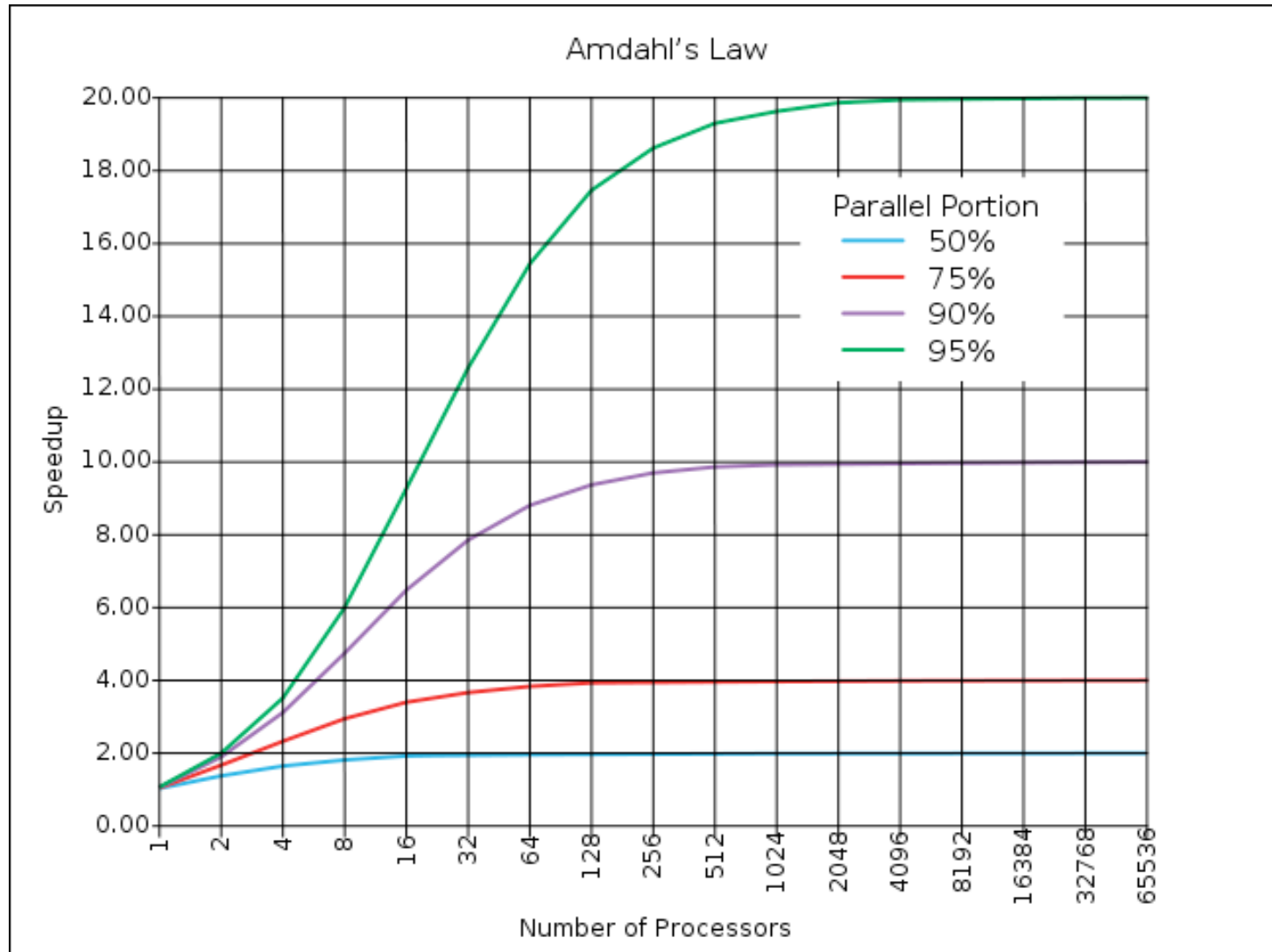
AMDAHL'S LAW

- Maximum speedup parallelizing a system:

$$S = \frac{1}{1 - P + \frac{P}{N}}$$

- P is the proportion of a program that can be made parallel
 - (1-P) is then the part that cannot be parallelized
- Theoretically maximum for P = 1 (linear speedup)
 - actually there are specific cases with $S > N$ (super-linear) speedup

AMDAHL'S LAW



REMARKS FROM AMDAHL'S LAW

- We can improve performances by adding processors only for those parts that can run in parallel, not for sequential parts
=> strong impact on performances (of sequential parts)
- But sequential parts are often needed for correctness (next module)

EFFICIENCY

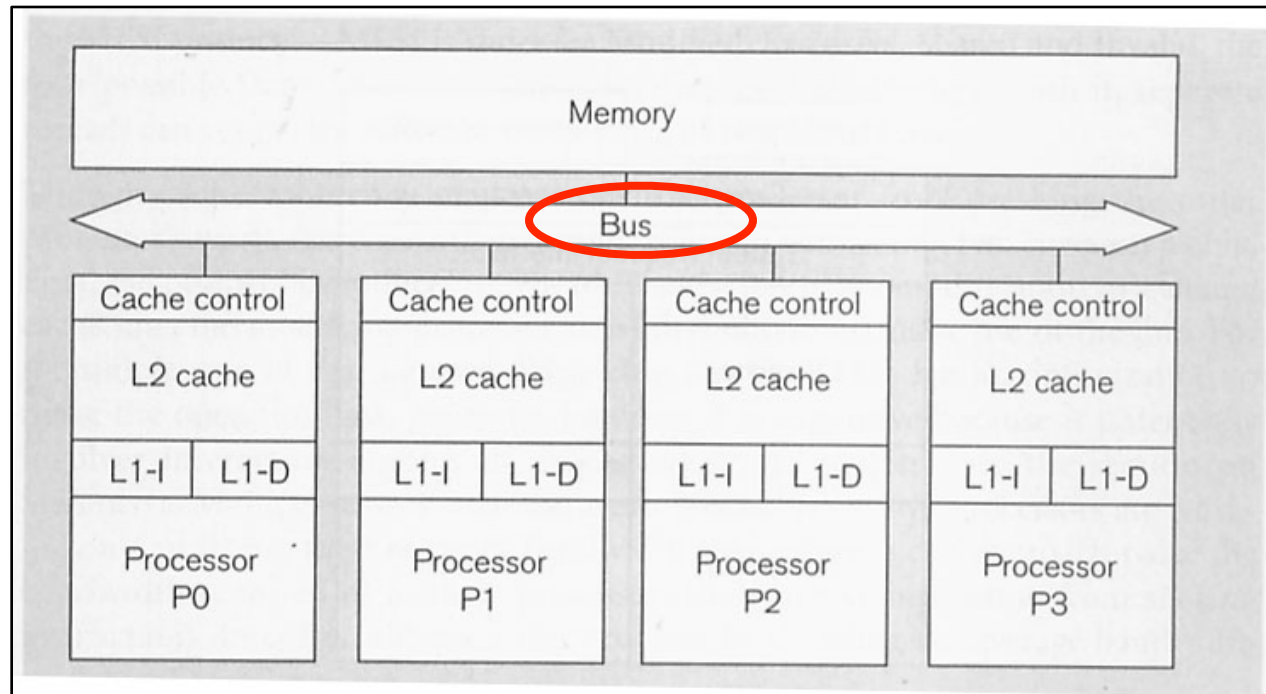
- Normalized measure of speed-up indicating how effectively each processor is used (S = speedup, N = number of processors)

$$E = \frac{S}{N}$$

- The ideal efficiency is 1 = all processors are used at full capacity
 - typically lower

A NEW BOTTLENECK: MEMORY

- Shared memory and bus as potential bottleneck
 - only one memory operation takes place at a time
 - importance of the cache
 - cache coherency protocol more and more complex and smart



CONCURRENT PROGRAMMING AS A PARADIGM

WHY CONCURRENT PROGRAMMING: DESIGN & ABSTRACTION

- Abstraction and engineering
 - define a proper level of abstraction for programs which interact with the environment, control multiple activities and handle multiple events..
 - objects from OOP are not enough
- Concurrency as a tool for software design and construction
 - rethinking to the way in which we solve problems
 - basic algorithms & data structures
 - rethinking to the way in which we design and build systems
 - new level of abstraction
 - different kind of decomposition, modularization, encapsulation
- Affecting the full engineering spectrum
 - **modelling, design, implementation, verification, testing**

CONCEPTS: PROCESSES

- **Processes** ~ a sequential program in execution
 - the basic unit of a concurrent system, single thread of control
 - logical thread of control / control flow, not necessarily physical
 - sequence of instructions operating together as a group
 - unit of work (task)
 - abstract / general concept
 - ...not necessarily related to OS processes
- *speed independence*
 - process execution is meant to be completely asynchronous with each other
 - we can't do any assumption about their speed
 - **non-determinism**

PROCESS INTERACTION

- **Process interaction**
 - any non trivial concurrent program is based on multiple processes that need to interact in some way
- Basic kinds of interaction:
 - **cooperation**
 - **competition** / contention
 - interferences

PROCESS INTERACTION: COOPERATION

- Refers to interactions which are both *expected* and *wanted*
 - they are part of the semantics of the concurrent program
- Two basic kinds
 - **communication**
 - concerns the need of realising an information flow among processes, typically realised in terms of messages
 - introduction of specific supports for the exchange of messages
 - **synchronisation**
 - concerns the explicit definition or presence of **temporal relationships** or dependencies among processes and among actions of distinct processes
 - introduction of specific supports for the exchange of temporal signals

PROCESS INTERACTION: CONTENTION / COMPETITION

- Refers to interactions which are *expected* and *necessary*, but *not wanted*
 - typically concerns the need of coordinating the access by multiple processes to shared resources
- Two basic class of problems
 - **mutual exclusion**
 - ruling the access to shared resources by distinct processes
 - **critical sections**
 - ruling the concurrent execution of blocks of actions by distinct processes

SYNCHRONISATION VS. MUTUAL EXCLUSION

- Different - even if related - concepts
 - “synchronisation = mutual exclusion urban legend” [BUH-05]
 - false story, still present in textbooks / research papers
 - synchronisation defines a *timing relationship* among processes
 - *maintaining time-relationships which includes actions happening at the same time or happening at the same relative rate or simply some action having to occur before another* (precedence relationships)
 - mutual-exclusion defines a *restriction* on access to shared data
 - mutual-exclusion is meaningless if no shared data is involved
- Relationships
 - mutual-exclusion typically require some forms of *implicit synchronisation*
 - blocking some actions, waiting for other actions to complete
 - synchronisation does not necessarily require any kind of shared data and the mutual exclusion

ON THE DIFFICULTY OF SYNCHRONISATION...

- “Alice and Bob live together, happily without cell-phones. Both are responsible to buy the milk when it finishes...”

Time	Alice	Bob
5:00	Arrive home	
5:05	Look in the fridge; no milk	
5:10	Leave for a grocery	
5:15		Arrive home
5:20		Look in the fridge; no milk
5:25	Buy milk	Leave for grocery
5:30	Arrive home; put milk in fridge	
5:40		Buy milk
5:45		Arrive home; oh no!

A SOLUTION: NOTES IN THE FRIDGE (1/2)

- Looking for a solution to ensure that:
 - only one person buys the milk, when there is no milk
 - someone always buys the milk, when there is no milk
- Tentative solution: using notes on the fridge!

PROGRAM for Alice & Bob

```
1 if (no note) then
2   if (no milk) then
3     leave note
4     buy milk
5     remove note
6   fi
7 fi
```

- “if you find that there is no milk and there is no note on the door of the fridge, then leave a note on the fridge’s door, go and buy milk, put the milk in the fridge, and remove your note.”
- Does it work? Not always actually...

A SOLUTION: NOTES IN THE FRIDGE (2/2)

(..NOT SO EASY, ACTUALLY..)

Time	Alice	Bob
5:00	Arrive home	
5:05	Look at the fridge; no note	
5:10	...ops! need a toilet	
5:15	...still at the toilet...	Arrive home
5:20	...still at the toilet...	Look at the fridge; no note
5:21	...still at the toilet...	Look in the fridge; no milk (argh)
5:22	...still at the toilet...	leave note
5:25	...still at the toilet...	go and buy milk
5:45	look in the fridge: no milk (*)	...
5:50	leave note...	

[*] Alice does not realize that a note was put on the fridge (she is not really a good observer) and strictly follows the established program

PROCESS INTERACTION: INTERFERENCES

- Refers to interactions which are *neither expected, nor wanted*
 - producing bad effects only when the ratio among the process speeds assumes specific values (time-dependent errors)
- The “nightmare” of concurrent programming
 - “heisen-bugs”
 - when debugging influence the bugs...

INTERFERENCES: RACE CONDITIONS

- **Race condition** or **race hazard** or simply **race**
 - whenever two or more processes concurrently access and update shared resources, and the result of the single update *depends on the specific order occurring in process access*
- Related to two main types of programming errors
 - bad management of expected interactions
 - presence of spurious interactions not expected in the problem

CRITICAL SITUATIONS

- Interferences and errors in concurrent programs can lead to *critical situations* for the concurrent system in the overall
 - **Deadlock** (...or deadly embrace (Dijkstra))
 - **Starvation** (or unfairness)
 - **Livelock**

DEADLOCK

- Situation wherein two or more competing actions (processes) are waiting for the other to finish, and thus neither ever does
 - typically concerns the release of a locked shared resource, the reception of a temporal signal or a message
- Remark
 - it concerns *multiple processes*

STARVATION

- Situation wherein a process is blocked in an infinite waiting
- Resource starvation
 - the process is perpetually denied in accessing necessary resources.
 - without those resources, the program can never finish its task
- Remark
 - it concerns a *process*

LIVELOCK

- A livelock is similar to a deadlock, except that the states of the processes involved in the livelock constantly change with regard to one another, none progressing
- Livelock is a special case of resource starvation
 - the general definition only states that a specific process is not progressing

CONCURRENT LANGUAGES AND MACHINES

- To *describe / specify* a concurrent program we need **concurrent programming languages**
 - enabling programmers to write down programs as set of instructions to be executed concurrently
- To *execute* a concurrent program we need a **concurrent machine**
 - a machine (which can be abstract) designed to handle the execution of multiple sequential processes, by exploiting multiple processors (physical or virtual)

CONCURRENT MACHINES

- A **concurrent machine** provides:
 - a support for the execution of concurrent programs and realizing then concurrent computations
 - as many virtual processors as the number of processes composing the concurrent computation
- Providing basic mechanisms for:
 - **multiprogramming**
 - virtual processors generation and management
 - **synchronisation** and **communication**
 - **access control** to resources

BASIC MECHANISMS

- **Multiprogramming**
 - set of mechanisms that make it possible to create new virtual processors and allocate physical processors of the lower-level machine to the virtual processors by means of scheduling algorithms
- **Synchronisation and Communication**
 - two different typologies of mechanisms, related to two different *architectural models* for concurrent machines:
 - **shared memory model**
 - presence of a shared memory among the virtual processors
 - example: multi-threaded programming
 - **message passing model**
 - every virtual processor has its own memory and no shared memory among processors is present
 - every communication and interaction among processors is realized through message passing

FROM MACHINES TO PROGRAMMING LANGUAGES

- Programming languages for specifying concurrent programs on top of concurrent machines
 - programs organised as sets of sequential processes to be executed concurrently on the virtual processors of the concurrent machine
- Basic constructs for
 - **specifying concurrency**
 - creation of multiple processes
 - **specifying process interaction**
 - synchronisation and communication
 - mutual exclusion

CONCURRENT PROGRAMMING LANGUAGES - DESIGN APPROACHES

- Three main design approaches
 - **sequential language + library with concurrent primitives**
 - e.g. C + PThreads
 - **language designed for concurrency**
 - e.g. OCCAM, ADA, Erlang, Go
 - **hybrid approach**
 - sequential paradigm extended with a native support for concurrency
 - e.g. Java, Scala
 - library and patterns based on basic mechanisms
 - e.g. `java.util.concurrent`

CONCURRENCY IN MAINSTREAM LANGUAGES

- For the most part, mainstream languages - both procedural (like C) and Object-Oriented (Java) - provide a support for the creation and execution of processes by means of libraries
 - without extending the language
 - not completely true for Java
- Support for multi-threaded programming
 - threads as implementation of the abstract notion of process
 - also called “lightweight processes” by referring to OS “heavyweight processes”
 - not to be confused with the notion of process as defined in OS
 - process as a programming in execution, with one or multiple control flows (threads)
- Main examples
 - multi-threaded programming in Java
 - Pthread library for C/C++ language on POSIX systems

BEYOND THREADS: RICH SCENARIO

- **Task-based frameworks**
 - e.g. Java Executor framework
- **Asynchronous Programming**
 - event-loop based models
 - AsyncTasks everywhere
- **Actors**
 - Concurrent OOP
 - *active objects* variant
- **Synchronous Message Passing**
 - Processes + synchronous message passing
- **Reactive Programming**
 - e.g. Functional Reactive Programming, Reactive extensions (Rx).
- **Parallel computing**
 - GPGPU programming
 - Lambda Architecture / Parallelism for BigData

BIBLIOGRAPHY

- **[SUT-12]**
 - Sutter's Mill. Herb Sutter on software, hardware, and concurrency. "Welcome to the Jungle". <http://herbsutter.com/welcome-to-the-jungle/>
- **[AND-83]**
 - Gregory Andrews and Fred Schneider - "Concepts and Notations for Concurrent Programming", *ACM Computing Surveys*, Vol. 15, No. 1, March 1983
- **[CLE-96]**
 - Rance Cleaveland, Scott Smolka et al - "Strategic Directions in concurrency Research", *ACM Computing Surveys*, Vol. 28, No. 4, Dec. 1996
- **[ROS-97]**
 - Roscoe, A. W. (1997). *The Theory and Practice of Concurrency*. Prentice Hall. ISBN 0-13-674409-5.
- **[BUH-05]**
 - Peter Buhr and Ashif Harji. "Concurrent Urban Legends". *Concurrency and Computation: Practice and Experience*. 2005. 17:1133-1172.